DESARROLLO Y EVALUACIÓN DE UNA APLICACIÓN CONCURRENTE MEDIANTE VIBECODING Y SPEC-DRIVEN DEVELOPMENT

Integrantes:

Marcos Agustín Caronte

mcaronte286@alumno.unlam.edu.ar

Fernando Nicolás Mallet

fmallet@alumno.unlam.edu.ar

Nicolás Pensel

npensel@alumno.unlam.edu.ar

Alejandro Diego Garay

agaray645@alumno.unlam.edu.ar

Asignatura:

(3900) Programación Concurrente

Fecha

21/10/2026

Resumen:

En esta actividad desarrollamos y evaluamos ReparaBarrio, un prototipo de escritorio orientada a la gestión operativa de un taller de reparaciones, donde múltiples órdenes de trabajo requieren recursos limitados y compartidos (como puestos de trabajo, técnicos y herramientas). Permite cargar y consultar órdenes, técnicos, puestos de trabajo y herramientas y evaluar qué reparaciones pueden iniciarse según la disponibilidad de los recursos y coordinar su asignación exclusiva, evitando que un mismo recurso sea utilizado simultáneamente por distintas órdenes. También permite realizar el seguimiento de los estados de las reparaciones y liberar los recursos cuando corresponde. Además, agrega un módulo de simulación independiente que reproduce la ejecución concurrente de múltiples reparaciones ficticias, esto hace que se pueda observar el comportamiento del sistema ante diferentes cargas de trabajo y situaciones de competencia por recursos.

Utilizamos Java 21, Codex CLI como agente de IA combinada con Specific-Driven Developement (SDD) como estrategia de desarrollo, con revisión humana de decisiones y resultados. La aplicación dispone de un modo operativo y un módulo de simulación que ejecuta trabajos ficticios concurrentes. Para estudiar el producto realizamos pruebas automatizadas y manuales, aplicamos rúbricas de funcionalidad y concurrencia, y medimos el uso de CPU, memoria y tokens. Pudimos observar resultados favorables en flujos de tipo “camino feliz” y escenarios de 100 órdenes, aunque también detectamos limitaciones en recuperación tras interrupciones y cobertura de pruebas. Presentamos los resultados con sus restricciones metodológicas, diferenciando el funcionamiento comprobado de los requisitos todavía no acreditados. El trabajo permite poner sobre la mesa tanto la utilidad de la generación asistida por IA como la necesidad de especificar, revisar y verificar críticamente el software obtenido.

Palabras Clave: *programación concurrente, generación de código asistida por IA, vibecoding, exclusión mutua, desarrollo dirigido por especificaciones, SDD, métricas de software*

I. CONTEXTO

Este trabajo se realizó para la Actividad Práctica Obligatoria N.º 1 de Programación Concurrente, correspondiente al segundo cuatrimestre de 2026 de la Universidad Nacional de La Matanza. La consigna propuso elaborar un programa concurrente mediante herramientas de generación de código a partir de prompts, analizar el resultado y documentarlo con métricas y referencias científicas. Nuestro grupo, M9, tomó como caso de estudio el desarrollo de una aplicación para un taller de reparaciones: un problema cercano a situaciones operativas reales, pero acotado a los objetivos académicos de la materia.

Denominamos *ReparaBarrio* al prototipo desarrollado. Nuestro propósito se centró en estudiar qué ocurre cuando varias órdenes compiten por recursos físicos limitados y cómo evitar asignaciones incompatibles. Se eligió una aplicación de escritorio local para Windows, con un modo operativo que usa persistencia y un modo de simulación independiente.

II. INTRODUCCIÓN

A. El problema de la concurrencia y los recursos compartidos

En un sistema concurrente, distintas actividades pueden intentar leer o modificar un mismo estado. Cuando las operaciones dependen de recursos compartidos, el orden de ejecución importa: dos solicitudes aparentemente válidas podrían intentar asignar una misma herramienta, producir una actualización parcial o responder con información que dejó de ser vigente. El problema de la exclusión mutua y la necesidad de proteger secciones críticas se encuentran entre los temas clásicos de la Programación Concurrente.

En *ReparaBarrio*, el caso crítico aparece al confirmar el inicio de una reparación. La consulta inicial solo propone una asignación; entre esa consulta y la confirmación, otra operación podría haber cambiado el inventario disponible. Por lo tanto, no alcanza con comprobar los recursos una sola vez: el sistema debe revisar el estado actualizado y confirmar o rechazar la operación como una unidad coherente, en forma atómica.

B. Desarrollo asistido por IA y necesidad de validación

Las herramientas de generación de código pueden acelerar la producción de prototipos y pruebas a partir de instrucciones expresadas en lenguaje natural. Sin embargo, la generación de una solución posible no implica que sea correcta en todos los escenarios. Hay estudios empíricos que analizaron tanto la corrección como la comprensibilidad del código generado y encontraron diferencias relevantes según el lenguaje y la tarea. En otros casos se llegaron a examinar riesgos de seguridad en código sugerido por un asistente de programación IA. Todo lo anterior refuerza la importancia de verificar los resultados antes de adoptarlos.

En nuestro caso, nuestra tarea fue, además de pedirle al agente que programara, necesitamos definir requisitos, revisar decisiones de diseño, ejecutar pruebas en nuestro entorno, detectar comportamientos incorrectos y decidir cuándo continuar o detener una implementación. Esta experiencia se documentó mediante un proceso de desarrollo dirigido por especificaciones (Spec-Driven Development, SDD).

C. Objetivo y alcance del estudio

Nos propusimos desarrollar un prototipo concurrente de cierta complejidad y evaluar sus capacidades y limitaciones. En concreto, analizamos: (a) el cumplimiento funcional; (b) la sincronización, las carreras, los bloqueos y la exclusión mutua; (c) el consumo observado de CPU y memoria en dos acciones; y (d) los prompts, refinamientos y tokens utilizados durante el desarrollo con Codex.

III. MÉTODOS

A. Requisitos y funcionamiento de ReparaBarrio

El sistema administra órdenes de reparación y recursos representados por técnicos, puestos y herramientas. Cada orden solicita un conjunto de recursos y atraviesa estados definidos: PENDIENTE, EN_CURSO, FINALIZADA, FALLIDA o CANCELADA. El planificador propone la primera orden factible por antigüedad de registro. Si una orden antigua no puede comenzar porque carece de recursos, se puede considerar una posterior; si una orden

antigua ya es factible, no corresponde saltearla arbitrariamente.

Una propuesta de inicio no equivale a una reserva, sino que recién cuando se confirma el inicio se comprueba nuevamente la disponibilidad y se asignan de manera conjunta los recursos físicos necesarios. La finalización o el fallo de una reparación libera sus recursos; la cancelación está prevista para órdenes pendientes. Se registran estados e historial para poder consultar qué ocurrió en el taller. Estas reglas se definieron en la especificación funcional del proyecto y se mantuvieron como referencia para el código y las pruebas.

El modo operativo utiliza persistencia local H2; el modo de simulación genera cargas ficticias con una semilla configurable, trabajadores concurrentes y un contexto aislado que modifica la información en el modo operativo. Esta separación nos permitió experimentar con órdenes, recursos y conflictos sin utilizar datos del taller real ni comprometer la base persistente. La simulación no representa reparaciones físicas, sino trabajos artificiales para observar el comportamiento concurrente.

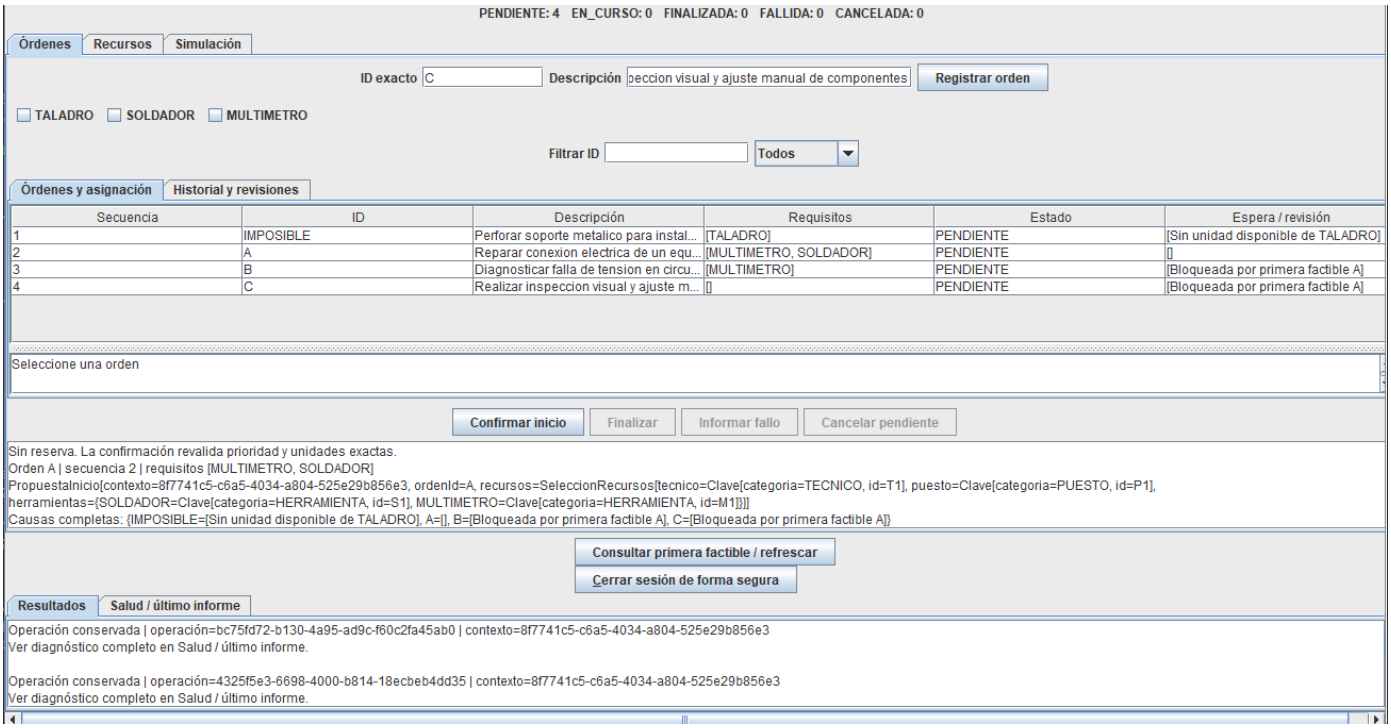


Fig. 1. MODO OPERATIVO - ÓRDENES

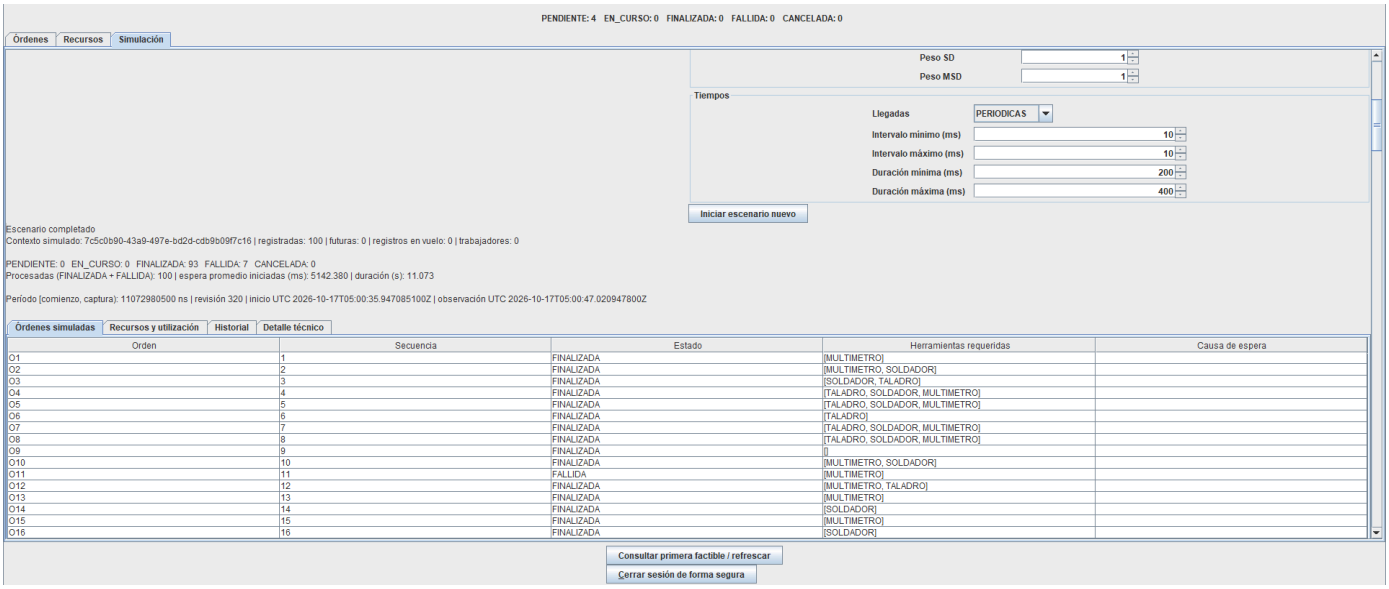


Fig. 2. MODO SIMULACIÓN

B. Arquitectura y mecanismos concurrentes

Trabajamos con Java 21, Maven, JUnit, Swing y una base H2 mediante JDBC. En el núcleo de aplicación se utilizó un "coordinador" común (mecanismo centralizado de sincronización) para proteger las operaciones sobre el estado del taller, con un bloqueo justo basado en ReentrantLock (es decir, un mutex utilizado para controlar acceso a secciones críticas). Las operaciones adquieren el coordinador antes de abrir una transacción atómica; luego consultan, validan, actualizan y confirman o revierten. En los escenarios normales, se busca que el resultado exitoso solo se informe después del commit. La base de datos y las validaciones del dominio complementan la protección de las asignaciones. El bloqueo se utiliza solamente durante las operaciones que requieren acceso exclusivo a los datos compartidos, como el inicio y la finalización de una reparación. Una vez asignados los recursos, las reparaciones simuladas (modo simulación) pueden ejecutarse de manera concurrente. El inicio y el cierre necesitan exclusión mutua, pero los trabajos ficticios pueden desarrollarse fuera de esa sección crítica. Así se pueden observar órdenes EN_CURSO simultáneas cuando cuentan con recursos independientes.

La implementación también nos permitió relacionar *ReparaBarrio* con distintos conceptos estudiados en la asignatura. El simulador utiliza varios hilos de ejecución y un conjunto configurable de trabajadores (*thread pool*) para procesar las reparaciones. Su funcionamiento presenta cierta semejanza con el problema productor-consumidor, ya que se generan órdenes que posteriormente son atendidas por trabajadores concurrentes. Sin embargo, no se trata de una implementación tradicional de ese patrón: las órdenes no se procesan simplemente según su llegada, sino que interviene un planificador que determina cuáles pueden comenzar según su antigüedad y la disponibilidad de recursos.

Una decisión importante de diseño fue limitar el alcance de la sección crítica: la exclusión mutua se aplica durante las operaciones que modifican el estado compartido, como la confirmación del inicio y la finalización de una reparación, pero no durante la ejecución del trabajo simulado. De esta manera, una vez asignados los recursos, distintas reparaciones ficticias pueden avanzar concurrentemente cuando disponen de recursos independientes. Esto permite observar varias órdenes en estado EN_CURSO al mismo tiempo, sin mantener bloqueadas innecesariamente otras operaciones del sistema.

TABLA. 1. ESQUEMA SIMPLIFICADO DE RESPONSABILIDADES DE *ReparaBarrio*

INTERFAZ Y MODOS	NÚCLEO COMPARTIDO	ESTADO Y EJECUCIÓN
Swing: Órdenes / Recursos	Servicio de aplicación y coordinador	H2/JDBC: modo operativo
Swing: Simulación	Reglas de dominio + planificador	H2 en memoria: simulación
Acciones humanas o automáticas	Confirmación atómica y validación	Trabajadores ficticios concurrentes (hilos reales de ejecución que realizan reparaciones simuladas)

C. Proceso de desarrollo con Codex y SDD

Organizamos el trabajo en especificación, planificación e implementación por hitos. Primero redactamos instrucciones para obtener *SPEC.md* y las sometimos a revisión. Después definimos un *PLAN.md* con decisiones de arquitectura, reglas y una secuencia de entregables. En los hitos iniciales se enfatizó más el dominio y la persistencia; posteriormente incorporamos las pruebas de concurrencia, la recuperación, la interfaz de usuario, el modo de simulación y la integración.

Codex CLI fue la herramienta que se utilizó para generar y modificar el código a partir de las instrucciones que dimos; el modelo registrado en las sesiones analizadas fue gpt-6.1-sol. Las decisiones de alcance y de aceptación, las pruebas ejecutadas fuera del entorno del agente y las operaciones Git/Maven realizadas manualmente se diferenciaron del trabajo delegado a Codex.

TABLA. 2. SECUENCIA DE TRABAJO UTILIZADA EN EL DESARROLLO

ESPECIFICAR	PLANIFICAR	IMPLEMENTAR Y VALIDAR
P01 → SPEC.md R01 → Refinamiento	P02 → PLAN.md R02 → Refinamiento	Prompts por hito (H1 - H7) → código y pruebas
Requisitos y criterios de aceptación	Arquitectura, riesgos, dependencias	Pruebas externas → Análisis → correcciones y nuevos prompts

Este proceso ayudó a que la aplicación converja de manera gradual hacia lo especificado, pero no eliminó los errores. En particular, durante el hito H4 (etapa relacionada con recuperación y arranque de la aplicación) las pruebas de recuperación mostraron fallos que no se resolvieron por completo. Por ello, el ciclo de revisión no consistía en aprobar automáticamente las respuestas del agente, sino en conservar también los resultados negativos y restringir el alcance como prototipo de investigación académica cuando la evidencia lo requería.

D. Validación

Aplicamos pruebas automatizadas con JUnit, pruebas concurrentes coordinadas mediante señales, ensayos externos sobre Windows y recorridos manuales de la UI. Se examinaron situaciones como doble intento de inicio, competencia por una herramienta exclusiva, trabajo simultáneo con recursos independientes, inicio frente a cancelación, cierre frente a fallo, transacciones y escenarios de simulación reproducibles.

Como parte de la validación, se ejecutaron tests y se establecieron criterios de aceptación. De esta manera se diferenciaba lo que había sido probado en un alcance acotado de la aplicación de aquellas verificaciones parciales o fallidas.

E. Rúbricas y métodos de medición

Para la evaluación cualitativa empleamos las dos rúbricas entregadas por la cátedra. La rúbrica de funcionalidad considera funcionamiento sin errores, cumplimiento de requisitos y diseño de interfaz. La de concurrencia considera sincronización, ausencia de deadlocks, ausencia de condiciones de carrera y exclusión mutua. En ambas consideramos niveles A, B o C, acompañados de una propuesta numérica de autoevaluación.

Para CPU y memoria realizamos una medición piloto del proceso Java con PowerShell y System.Diagnostics.Process. Seleccionamos dos acciones: registro manual de cuatro órdenes y ejecución de una simulación de 100 órdenes. La medición se efectuó en Windows con 16 procesadores lógicos y una sola repetición de cada acción. El porcentaje de CPU se normalizó tomando como 100 % la capacidad agregada de los 16 procesadores lógicos. La memoria residente del proceso se obtuvo mediante WorkingSet64 y se convirtió de MiB a MB decimales.

El uso de CPU se calculó como 100 × (incremento de tiempo de CPU del proceso / tiempo real transcurrido) / 16. Los intervalos de medición se iniciaron y finalizaron manualmente mediante el script de PowerShell. Por lo tanto, los promedios obtenidos incluyen no solo el procesamiento de la aplicación, sino también los tiempos destinados al ingreso de datos por teclado, la interacción con la interfaz y la espera de resultados. En consecuencia, estas mediciones reflejan el consumo de recursos durante el uso de la aplicación y no exclusivamente el rendimiento del algoritmo.

Para analizar el consumo de tokens durante el desarrollo, conservamos los registros de 35 sesiones de Codex. De ellas, 34 incluían información sobre el consumo total de tokens, mientras que en una sesión ese dato no estaba disponible. Por separado, identificamos 35 mensajes de instrucciones enviados explícitamente al agente y contabilizamos los tokens contenidos en esos textos. En este último cálculo no incluimos las instrucciones generales del proyecto definidas en el archivo AGENTS.md, que Codex incorpora automáticamente como contexto de trabajo.

IV. RESULTADOS Y OBJETIVOS

A. Funcionamiento observado y pruebas

En las verificaciones manuales se pudieron registrar órdenes y recursos, consultar propuestas, confirmar inicios y observar estados de trabajo. También se ejecutaron simulaciones con carga configurable. Una corrida documentada de 100 órdenes terminó con 93 FINALIZADAS y 7 FALLIDAS, sin órdenes pendientes ni trabajos en curso. Esos siete fallos corresponden a resultados previstos de las reparaciones ficticias y no representan siete errores del programa.

Durante el hito H6, relacionado con las pruebas automatizadas específicas del modo de simulación, se ejecutaron 141 pruebas que finalizaron satisfactoriamente y permitieron verificar determinados componentes y escenarios del núcleo de la aplicación y del simulador. Sin embargo, estos resultados no permiten concluir por sí solos que toda la aplicación cumpla la totalidad de los criterios de aceptación.

La evaluación de los criterios de aceptación permitió identificar funcionalidades verificadas satisfactoriamente en escenarios específicos, junto con otras cuya validación resultó parcial o presentó dificultades. Las pruebas posteriores del paquete portable aportaron evidencia favorable sobre su funcionamiento, aunque no permitieron acreditar el cumplimiento integral de todos los criterios. Por otra parte, durante el hito H7, dedicado a las pruebas de integración y regresión, se ejecutó una selección de 146 pruebas, de las cuales 82 fueron satisfactorias, 60 fallaron, una presentó un error y tres fueron omitidas. Varias de las pruebas fallidas estuvieron relacionadas con una precondición de apertura del almacenamiento H2, por lo que estos resultados no deben interpretarse necesariamente como 60 defectos funcionales independientes.

B. Autoevaluación de funcionalidad y concurrencia

TABLA 3. AUTOEVALUACIÓN DE FUNCIONALIDAD (PROPUESTA DEL GRUPO)

Criterio	Nivel	Puntaje propuesto
Funciona sin errores	B	15 / 40
Cumplimiento del objetivo funcional	B	20 / 40
Diseño de la aplicación	B	8 / 20
TOTAL	—	43 / 100

- Respecto al criterio Funciona sin errores (15/40): La aplicación compila y permite realizar las principales operaciones previstas, como registrar órdenes y recursos, consultar propuestas de asignación, iniciar y finalizar reparaciones y ejecutar simulaciones concurrentes. Estas funcionalidades fueron verificadas mediante pruebas automáticas y manuales. Sin embargo, durante el desarrollo encontramos problemas que no se resolvieron completamente, especialmente en la recuperación de información después de interrupciones inesperadas y en algunos escenarios de ejecución concurrente. Por este motivo, no consideramos que el funcionamiento esté libre de errores y asignamos 15 puntos dentro del nivel B, de esta manera se contemplan los resultados obtenidos como las limitaciones pendientes.

- Respecto del Cumplimiento del objetivo funcional (20/40): Consideramos que *ReparaBarrio* cumple parcialmente los objetivos planteados. Logramos implementar las funcionalidades centrales de gestión de órdenes, planificación y asignación exclusiva de recursos, junto con un módulo capaz de ejecutar simulaciones concurrentes de 100 órdenes. Sin embargo, algunos requisitos relacionados con la recuperación de sesiones, la persistencia ante fallos y la validación integral del sistema no se cumplieron o no se verificaron completamente. Por ello, asignamos el nivel B con 20 puntos, ya que se alcanzó una parte importante del objetivo original, pero no la totalidad de los requisitos definidos.
- Respecto del Diseño de la aplicación (8/20): La interfaz gráfica organiza las funcionalidades en tres secciones principales: Órdenes, Recursos y Simulación. Esta distribución permite identificar las operaciones disponibles, ingresar información y consultar los resultados. Durante las pruebas pudimos utilizarla para realizar las tareas previstas. Sin embargo, observamos aspectos mejorables, principalmente en la distribución de los controles, la cantidad de parámetros necesarios para configurar las simulaciones y la claridad de cierta información técnica presentada al usuario. Por estas razones, consideramos que la interfaz es funcional, aunque todavía podría resultar más sencilla e intuitiva, y asignamos 8 puntos dentro del nivel B.

Como evaluación general, a partir de estos criterios, nuestra autoevaluación de funcionalidad alcanza 43 puntos sobre 100. Esta calificación refleja que se logró desarrollar una aplicación operativa en sus funciones principales, pero que todavía presenta limitaciones que impiden considerarla completamente validada de manera exhaustiva.

TABLA 4. AUTOEVALUACIÓN DE CONCURRENCIA (PROPUESTA DEL GRUPO)

Criterio	Nivel	Puntaje propuesto
Sincronización correcta	B	15 / 25
Ausencia de deadlocks	B	15 / 25
Ausencia de condiciones de carrera	B	15 / 25
Uso correcto de exclusión mutua	B	15 / 25
TOTAL	—	60 / 100

- Respecto al criterio Sincronización correcta (15/25): *ReparaBarrio* incorpora mecanismos de sincronización para coordinar las operaciones concurrentes que modifican el estado compartido del taller. Mediante un coordinador centralizado, se busca mantener la consistencia de las órdenes y de los recursos durante operaciones como el inicio, la finalización y la cancelación de reparaciones. Las pruebas realizadas mostraron resultados satisfactorios en situaciones habituales y en diversos escenarios concurrentes. Sin embargo, todavía existen verificaciones pendientes relacionadas con cierres simultáneos y situaciones excepcionales. Por ello, consideramos que la sincronización funciona en los escenarios comprobados, aunque no se ha acreditado su comportamiento en todas las circunstancias, y asignamos 15 puntos dentro del nivel B.
- Respecto al criterio Ausencia de deadlocks (15/25)**:** Durante las pruebas realizadas no se observaron situaciones en las que los hilos quedaran bloqueados permanentemente entre sí, impidiendo el avance de las operaciones. Como evidencia, las simulaciones de 100 órdenes finalizaron sin trabajos detenidos indefinidamente. Además, el diseño utiliza un coordinador centralizado y evita mantener el bloqueo durante toda la ejecución de las reparaciones ficticias. Pero estos resultados no permiten descartar por completo la posibilidad de deadlocks en circunstancias no verificadas, especialmente ante interrupciones o problemas de persistencia. Por esta razón, asignamos 15 puntos dentro del nivel B.
- Respecto al criterio Ausencia de condiciones de carrera (15/25)**:** Para evaluar este criterio se realizaron pruebas en las que distintas operaciones competían por modificar una misma orden o utilizar recursos compartidos. Entre los escenarios examinados se incluyeron

intentos simultáneos de iniciar una reparación, competir por una herramienta exclusiva y finalizar o cancelar operaciones. En los casos verificados, se mantuvo la consistencia esperada. Sin embargo, durante el desarrollo se registró un comportamiento intermitente en una prueba relacionada con la finalización, el fallo y la reasignación de recursos. Aunque esto no demuestra por sí solo la existencia de una condición de carrera, constituye un resultado que requiere investigación adicional. Por ello, asignamos 15 puntos dentro del nivel B.

- Respecto al criterio Uso correcto de exclusión mutua (15/25)**:** Como se mencionó anteriormente, la aplicación utiliza un bloqueo explícito mediante ReentrantLock (mutex) para proteger las secciones críticas, especialmente durante la asignación y liberación de técnicos, puestos y herramientas. De esta manera, se busca garantizar que un recurso exclusivo no sea asignado simultáneamente a distintas órdenes y que las asignaciones se realicen de forma completa, evitando reservas parciales. Una decisión importante fue mantener estas secciones críticas acotadas: las reparaciones ficticias se ejecutan fuera del bloqueo, permitiendo que trabajos independientes avancen concurrentemente. Si bien las pruebas aportaron evidencia favorable sobre este mecanismo, su comportamiento ante determinadas situaciones excepcionales y de persistencia todavía requiere validación. Por ello, asignamos 15 puntos dentro del nivel B.

Nuestra autoevaluación de concurrencia alcanza 60 puntos sobre 100, correspondientes al nivel B en los cuatro criterios. Los resultados obtenidos muestran que *ReparaBarrio* incorpora mecanismos de coordinación y exclusión mutua que permiten ejecutar tareas concurrentes y preservar la consistencia del estado compartido en los escenarios evaluados. No obstante, las verificaciones pendientes y los comportamientos excepcionales detectados impiden considerar demostrada la corrección integral de la aplicación. Por este motivo, la puntuación refleja tanto las capacidades comprobadas como las limitaciones identificadas durante su validación.

C. Resultados del piloto de CPU y memoria

TABLA 6. CPU Y MEMORIA OBSERVADAS EN DOS ACCIONES

Magnitud	**Acción 1: 4 órdenes**	**Acción 2: 100 órdenes**
Duración muestreada	98,749 s	27,219 s
Cantidad de muestras	453	125
CPU media, equipo completo	0,3283 %	6,8993 %
CPU máxima por muestra	10,7714 %	54,1828 %
Memoria residente media	272,11 MB	487,14 MB

La simulación presentó mayor consumo medio de CPU y memoria residente que el registro manual de cuatro órdenes, algo compatible con la mayor actividad de trabajadores y procesamiento. Reiteramos que hay que tener en cuenta que los períodos de observación contienen tiempos de interacción con la aplicación y no equivalen necesariamente a los segundos efectivos de cálculo. Estos datos describen el comportamiento observado en nuestro equipo (16 CPUs lógicas y aproximadamente 34.222,30 MB de RAM física).

D. Tokens y refinamientos del desarrollo

TABLA 5. INSTRUCCIONES Y CONSUMO DOCUMENTADO DE CODEX

Indicador	**Valor**	**Alcance**
Ejecuciones identificadas	35	35 sesiones vinculadas
Mensajes específicos enviados	35	34 textos diferentes
Tokens de esos mensajes	47.577	js-tiktoken / o200k_base
Refinamientos documentados	10	Criterio de cambio importante
Consumo global informado	59.164.219	34 de 35 registros de datos de consumo (campo “usage”)

Para la planilla de métricas optamos por informar 47.577 tokens de los mensajes específicos enviados, porque la consigna se centra en el uso de prompts. No incluye los documentos Markdown que Codex leyó después de recibir instrucciones para hacerlo.

La auditoría alternativa de registros Codex suma 59.164.219 tokens de entrada y salida, incluidos tokens de contexto reutilizados mediante caché. La cobertura es 34/35 porque una ejecución no informó contadores. Mantenemos esta medida como

información técnica complementaria y aclaramos que el tokenizador o200k_base no está acreditado como idéntico al empleado internamente por el modelo.

En cuanto a los refinamientos, documentamos 10 reformulaciones importantes, incluyendo ajustes formales sobre SPEC/PLAN/diseños e instrucciones correctivas posteriores a pruebas.

E. Impresiones (enfoque cualitativo) del código generado

Abordaje y arquitectura: En nuestra evaluación consideramos que la arquitectura generada por Codex resulta apropiada para el problema planteado. La organización del sistema en módulos con responsabilidades diferenciadas permite separar las reglas de negocio, la planificación, la persistencia, la interfaz gráfica y la simulación. También valoramos que ambas modalidades utilicen las mismas reglas para decidir la asignación de recursos. Sin embargo, observamos que la coordinación centralizada como se mencionó anteriormente, si bien facilita mantener la consistencia de los datos, podría generar esperas cuando varias operaciones necesitan acceder a la misma sección crítica. Además, la incorporación de mecanismos de persistencia y recuperación aumentó considerablemente la complejidad del proyecto, especialmente en los escenarios que no logramos validar completamente.

Calidad del código: Desde el punto de vista de la organización y claridad, encontramos aspectos favorables en la separación de responsabilidades y en la posibilidad de probar distintos componentes mediante pruebas automatizadas. Esto facilitó identificar problemas y revisar partes concretas del sistema durante el desarrollo. Aunque hay que destacar, que una estructura aparentemente ordenada y una cantidad importante de pruebas satisfactorias no garantizan por sí solas la calidad del código. Los errores detectados y las dificultades para resolver determinados escenarios muestran que todavía existen aspectos que requieren revisión. Nuestra valoración es cualitativa relativamente superficial, ya que no realizamos un análisis estático integral ni calculamos métricas específicas de mantenibilidad o complejidad del código generado.

Seguridad y confiabilidad: En cuanto a la protección de la información, identificamos mecanismos orientados a preservar la integridad de los datos, como la validación de operaciones, la asignación exclusiva de recursos y el uso de transacciones para confirmar o revertir cambios. También se buscó mantener separados los datos reales del modo operativo de los utilizados durante las simulaciones. Desde ya que estas medidas no completan una evaluación exhaustiva del aspecto de la seguridad. No realizamos pruebas específicas de vulnerabilidades dado que quedó fuera del alcance previsto. A esto se suman las dificultades encontradas en la recuperación de información ante interrupciones inesperadas, que representan una limitación importante para la confiabilidad de la aplicación.

F. Limitaciones encontradas y pendientes

Una de las principales limitaciones pendientes es la recuperación de información después de interrupciones inesperadas, especialmente cuando no puede determinarse si una operación llegó a guardarse correctamente. También quedaron incompletas algunas verificaciones de integración, de funcionamiento de la interfaz bajo diferentes condiciones de uso y de cierre de la aplicación durante la ejecución de trabajos. Si este trabajo tuviera una continuidad más allá del alcance de este trabajo, completar estas pruebas, revisar los problemas detectados y profundizar el análisis de calidad y seguridad del código serían los principales focos antes de plantear su utilización fuera del ámbito académico.

V. CONCLUSIONES

La experiencia con *ReparaBarrio* nos permitió comprobar que es posible avanzar en una aplicación concurrente de cierta complejidad utilizando generación de código mediante prompts (vibecoding), siempre que exista un trabajo previo de especificación y una revisión constante del resultado. El agente contribuyó a producir módulos, pruebas e integración, pero las decisiones sobre alcance, los criterios de aceptación y la evaluación de fallos siguieron requiriendo intervención humana.

La aplicación consiguió demostrar recorridos nominales (“camino feliz”) y simulaciones con órdenes concurrentes, recursos exclusivos y resultados observables. Al mismo tiempo, las dificultades que encontramos en recuperación muestran que una implementación extensa y una batería de pruebas favorable no equivalen automáticamente a un sistema funcionalmente correcto en su totalidad. La distinción entre pruebas realizadas, pruebas bloqueadas por el entorno y pruebas efectivamente fallidas nos permitió poner en mejor perspectiva la aplicación sin sobrevalorarla.

Las métricas también necesitaron decisiones metodológicas. Los promedios de CPU y memoria

obtenidos describen una ejecución piloto concreta; asimismo, los tokens de mensajes enviados y el consumo global del agente representan magnitudes diferentes. Nos pareció importante no solo presentar las cifras sino sus definiciones y limitaciones eventualmente.

Desde el análisis del código generado, también observamos que la IA pudo producir una solución con una organización razonable y mecanismos de concurrencia adecuados para los escenarios principales. Además de elegirse correctamente un mecanismo de sincronización deben considerarse la complejidad de la implementación, la recuperación ante fallos, la mantenibilidad y la seguridad. En este sentido, entendemos que la revisión humana y las pruebas no son solamente una etapa final, sino una parte necesaria del desarrollo "vibecoding".

En síntesis, valoramos la utilidad del desarrollo asistido por IA para explorar soluciones y organizar un prototipo de programación concurrente. Nuestra principal conclusión es que la especificación, la verificación y el juicio crítico del equipo no se reemplazan por la generación automática de código. El resultado alcanzado es apropiado como caso de estudio académico con limitaciones explícitas, no como sistema listo para producción.

VI. REFERENCIAS Y BIBLIOGRAFÍA

A. Referencias bibliográficas

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts (10th ed.)*. Wiley.

[2] Stallings, W. (2015). *Operating Systems: Internals and Design Principles (8th ed.)*. Pearson.

B. Documentación del proyecto y material de cátedra

Material de cátedra: UNLaM, "APO1: Métricas" y "Plantilla de Actividades Prácticas" (2026). Documentación interna del Grupo M9: SPEC.md v0.2, PLAN.md v0.2, reportes H4/H6/H7, matriz de aceptación, rúbricas y registros de métricas PowerShell y Codex (octubre de 2026).